SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	**Total Marks:**	100
CO5 Statement:	*Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.*		
Student Name:	JAGADISH.V	**Reg. No.:**	192521352
Section / Batch:		**Date:**	

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

Q1 Permutation Generation Problem

Write a pseudocode to generate all possible permutations of a given set of elements using Backtracking. Clearly define inputs (set of elements) and output (all permutations). Design a complete algorithm with proper swapping and backtracking steps. Use loops and recursive calls effectively. Present the pseudocode in a clear and structured format. Ensure all permutations are generated without repetition.

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm Design	30%				
Use of Control Structures	20%				
Pseudocode Structure	10%				
Completeness	20%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Algorithm: Permutation Generation Using Backtracking

Step-by-Step Algorithm

1. **Start**
2. Read the input list elements.
3. If the list has **0 or 1 element**, return the list.
4. Create an empty list result.
5. Select each element one by one as the **current element**.
6. Remove the current element from the list to form remaining.
7. Recursively generate permutations of remaining.
8. Add the current element to the beginning of every generated permutation.
9. Store each permutation in result.
10. Repeat until every element has been selected as the current element.
11. Return result.
12. **Stop.**

Pseudocode

```
Algorithm GeneratePermutations(elements)

    if length(elements) <= 1 then
        return [elements]

    result ← empty list

    for i ← 0 to length(elements) - 1 do

        current ← elements[i]

        remaining ← elements[0:i] + elements[i+1:]

        permutations ← GeneratePermutations(remaining)

        for each p in permutations do
            result.append([current] + p)

    return result

End Algorithm
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Jagadish V
192521352RC

Q1
Permutation Generation Problem
100 marks

0/1 answered

Write a pseudocode to generate all possible permutations of a given set of elements using Backtracking. Clearly define inputs (set of elements) and output (all permutations). Design a complete algorithm with proper swapping and backtracking steps. Use loops and recursive calls effectively. Present the pseudocode in a clear and structured format. Ensure all permutations are generated without repetition.

Your Code

```
1  def get_permutations(elements):
2      # Base case: if list is empty or has one element, return it as a li
3      if len(elements) <= 1:
4          return [elements]
5
6      result = []
7      for i in range(len(elements)):
8          # Extract the current element
9          current = elements[i]
10         # The remaining elements after removing the current one
11         remaining = elements[:i] + elements[i+1:]
12
13         # Recursively find permutations of the remaining elements
14         for p in get_permutations(remaining):
15             result.append([current] + p)
16
17     return result
18
19 # Input
```

Input

stdin input

Output

```
Input: [1, 2, 3]
Output: [[1, 2, 3], [1, 3, 2], [2,
1, 3], [2, 3, 1], [3, 1, 2], [3, 2,
1]]
```